Introduction to Rust & eBPF with Rust

[Pulumati Ram]

REALTEK AI: Changing the Future

Introduction to Rust & eBPF with Rust

PukunatiRam

2026-05-28

Realtek SemiConductor Corporation

> Disclaimer <

Focus on systems evolution, not a language war

- Rust is not here to replace C everywhere -
- Complementary systems programming tool -
- focus on spatial/temporal memory safety bugs. -

#1

Rust as a systems programming language

zero-cost abstraction, borrow checker, memory layout ctrl, fearless concurrency, Rust in Linux kernel

#2

eBPF programming with Rust

Aya framework, observability case study

Agenda

1. Introduction to Rust: 2
2. How Rust Fits Systems Programming 6
3. Important Language features: 10
4. Borrowing & Lifetimes — Preventing Data Races 14
5. The Type System as a Security Tool 18
6. Other Key Security-Focused Features 24
7. Compiler Checks : Beyond Memory safety: 28
8. The Broader Rust Ecosystem 30

1. Introduction to Rust:

A Systems Programmer's Perspective:

· Memory Safety · Compiler Guarantees · Modern Concepts · performance

Systems software: Controls hardware and mediates between it and everything else.

It's characterised by three constraints which are its requirements:

1. **Direct hardware access**: memory-mapped registers, DMA engines, interrupt controllers
2. **No managed runtime**: no garbage collector, no VM, no OS safety net beneath you
3. **Correctness is load-bearing**: A bug does not crash one user session; it crashes the whole system, corrupts flash, or silently misdelivers data to hardware

Your daily work is systems programming:

- Peripheral drivers (PCIe, MIPI, USB, UART, I2C, SPI)
- DMA engine bring-up, scatter-gather list management
- Power management (DVFS, clock trees, voltage domains)
- Boot firmware, secure monitor.
- Android HAL native layer, Binder IPC, ION/DMA heap management

The languages that have historically owned this domain

Language	Domain	Memory safe?	No GC?
Assembly	firmware, boot	✗	✓
C	OS, drivers, embedded	✗	✓
C++	firmware, RTOS, Android HAL	✗ [1]	✓
Go	tooling, cloud	✓	✗
Rust	**all of the above**	✓	✓

[1] RAII helps; raw pointers escape freely.

Rust is the first language to occupy the **top-right cell simultaneously**, giving memory safety **and** no GC — with a formally verified type system. *Jung et al., RustBelt, POPL 2018*

Industry-wide memory safety statistics

Source	Finding
Microsoft MSRC	70 % of all CVEs are memory safety bugs
Chrome team	70 % of high-severity bugs are memory safety
Linux kernel	67 % of CVEs are memory safety violations
Android team	Memory unsafety estimated at $68B in security costs
NSA guidance	C and C++ flagged as “memory-unsafe” languages
CISA	“The Case for Memory Safe Roadmaps” — mandates shift

- These safety stats haven’t changes in 20 years.

Microsoft MSRC 2019; Google Project Zero 2020; Linux Security Summit 2019; CISA 2023

The bug classes that drive these numbers

- **Use-after-free (UAF)**: most common kernel CVE class: IRQ handler retains a pointer to freed `struct device`
- **Buffer overflow / OOB write**: DMA descriptor overrun corrupts adjacent kernel data
- **Null dereference**: `container_of()` result not checked, dereferenced in probe path
- **Data race**: two CPUs write a shared DMA counter without a lock
- **Uninitialised read**: `struct` field read before all error paths initialise it
- **Integer overflow**: `nents * sizeof(entry)` wraps; under-allocated scatter-gather list

- These are not exotic bugs requiring complex/adversarial inputs, they are everyday bugs we also encounter while de-bugging during SoC bring-up.
- KASAN, KCSAN, and lockdep catch them at runtime
- **Rust prevents them at compile time.**

Fix for the status quo:

> The only way to eliminate a class of bugs is to make them un-representable in the type system.

- Rust **eliminates every row in this table** at compile time, with zero runtime overhead.
- Rust meets all the systems programming requirements.
  - 3rd Programming model. (Memory Management Via Ownership & Borrowing)
  - type-system, compiler.

2. How Rust Fits Systems Programming

Property 1: No garbage collector, no runtime

Rust has no GC, no reference-counting runtime, no background threads, no stop-the-world pause. The memory model is:

- Stack allocation: zero overhead — exactly like C `int x;`
- Heap allocation: explicit, backed by the allocator you choose
- Destructor: called at a **statically known point** by the compiler, not by a runtime at an unpredictable time

This means Rust code can run:

- In interrupt handlers (ISR context)
- In firmware before the MMU is enabled
- In a `#![no_std]` kernel module with no OS beneath it
- On a bare-metal microcontroller with 16 KB of RAM

The binary output is a standard ELF — same format as a C object file. A Rust kernel module is a `.ko` that `insmod`, `lsmod`, and `rmmod` treat identically.

Property 2: Direct hardware access

Rust can do everything C can at the hardware interface:

```
// Memory-mapped I/O register write
// (identical to: *(volatile u32*)CTRL_REG = 0x1;)
unsafe {
    core::ptr::write_volatile(
        CTRL_REG as *mut u32, 0x1
    );
}

// Inline assembly — same as GNU C __asm__ __volatile__
use core::arch::asm;
unsafe {
    asm!("dsb sy", options(nostack));
}

// Raw pointer arithmetic — same as C, explicitly unsafe
let ptr = base_addr as *mut u32;
unsafe { *ptr.add(offset) = value; }
```

`unsafe { }` is not "turn off Rust" — it is an **explicit declaration** that you are taking responsibility for the invariants the type system cannot verify. It is grep-able, auditable, and contained.

Property 3: Zero-cost abstractions

The Stroustrup principle, applied

> "What you don't use, you don't pay for. What you do use, you couldn't hand-code any better."

Rust's abstractions compile to the same machine code as the equivalent hand-written C. This is not a promise — it is verifiable on Compiler Explorer.

Iterators and closures — no overhead

```
// High-level, expressive:
let total: u64 = latencies
  .iter()
  .filter(|&&ns| ns > threshold)
  .sum();

// Compiles to exactly this loop — same as C:
let mut total: u64 = 0;
for &ns in &latencies {
    if ns > threshold { total += ns; }
}
// Godbolt: identical ADDQ loop in both cases
```

Compiler Explorer — godbolt.org; rustc -O2

Generics — monomorphisation, not boxing

```
// One generic function:
fn min_of<T: Ord>(a: T, b: T) -> T {
    if a <= b { a } else { b }
}
// Compiler generates TWO specialised versions:
// min_of::<u32>(a: u32, b: u32) -> u32
// min_of::<u64>(a: u64, b: u64) -> u64
// No vtable. No boxing. No indirection.
```

Lifetime annotations are erased before codegen

```
// 'a is compile-time analysis only — zero runtime cost:
fn longest<'a>(x: &'a [u8], y: &'a [u8]) -> &'a [u8] {
  if x.len() >= y.len() { x } else { y }
}
// Generates: same two-compare, one-return instruction
//            sequence as the C pointer version
```

> Borrow checking, lifetime analysis, type inference — all stripped before LLVM sees the code. **The runtime binary is as lean as hand-written C.**

This is the often-overlooked performance **advantage** Rust has **over** C.

The C aliasing problem

C's pointer aliasing rules (C99 §6.5) say two pointers of different types **may not** alias — but two `u8*` pointers **might** always alias. The compiler must assume they overlap.

```c
// C: compiler cannot vectorise safely
// because it cannot prove src ≠ dst
void process(uint8_t *dst,
    const uint8_t *src, size_t n) {
      for (size_t i = 0; i < n; i++)
        dst[i] = src[i] | 0x80;
      }
// Must add `restrict` keyword AND trust the caller
void process(uint8_t *restrict dst,
         const uint8_t *restrict src, size_t n);
// restrict is a promise, not a proof
```

Rust's aliasing proof

Rust's exclusivity rule (`&mut T` is exclusive — no other reference exists) **proves** at compile time that `out` and `in_buf` do not overlap. LLVM gets this information and auto-vectorises without any annotation.

```rust
// Rust: exclusive borrow PROVES no overlap
// Compiler auto-vectorises — no annotation needed
fn process(out: &mut [u8], in_buf: &[u8]) {
    for (o, &b) in out.iter_mut().zip(in_buf) {
        *o = b | 0x80;
    }
}
// Generated: VPOR ymm loop (AVX2)
// No restrict. No trust. The type system proved it.
```

> The **same property** that prevents data races also enables better codegen. Safety and performance arise from the same source: the aliasing proof in the type system.

3. Important Language features:

- Ownership
- Borrowing
- Lifetimes
- Thread Safety

The three ownership rules

- **Rule 1 : Each value has exactly one owner**
- **Rule 2 : There can only be one owner at a time (ownership can be moved).**

C : compiles, crashes or corrupts silently

```
let s1 = String::from("hello"); // s1 owns the heap data
let s2 = s1;          // ownership MOVES to s2
// println!("{}", s1); // ← compile error: s1 was moved
```

The compiler tracks ownership **statically**. No runtime book keeping.

- **Rule 3 : When the owner goes out of scope, the value is dropped**

C : compiles, crashes or corrupts silently

```
{
    let buf = alloc_dma_buf(); // allocation
}   // ← Drop::drop() called HERE, automatically
    // No free() to forget. No leak possible.
```

Rust's ownership system it's a type theoretic answer to manual memory management.

- It's a memory management model where each value has a single owner at a time, and the Compiler enforces rules about ownership/borrowing/lifetimes.
- Shifts memory safety from a runtime concern (like GC) to a compile time verification handled by the type system.

-> Shifts memory safety from a runtime concern (like garbage collection) to a compile-time verification problem handled by the type system.

Most common kernel CVE class : caught at compile time in Rust.

- This is not a bug detection problem.
- This is program expressiveness problem.

C : compiles, crashes or corrupts silently

```c
struct dma_buf *buf = dma_alloc(dev, size);
submit_dma(dev, buf);

kfree(buf);           /* freed */

/* … 500 lines later … */
read_completion(buf); /* UAF — undefined behaviour */
/* gcc/clang: no warning, no error */
```

Rust : compile error, zero runtime cost

```rust
let buf = DmaBuf::alloc(dev, size)?;
submit_dma(dev, &buf);

drop(buf);            // buf is freed here

// read_completion(&buf);
// ^^^ error[E0382]: use of moved value: `buf`
//     value used here after move
//     |  drop(buf);
//     |       --- value moved here
```

- The error message names the **exact line** where the value was moved and the **exact line** of the illegal use, before the code ever runs.
- no `valgrind`, no `KASAN`, no `OEM` execution.

In C, every error exit path in a driver must remember to call the right cleanup.

Rust makes this structurally impossible to get wrong.

C : common leak pattern

```c
int my_driver_probe(struct pci_dev *pdev) {
    void *res_a = alloc_a();
    if (!res_a) return -ENOMEM;

    void *res_b = alloc_b();
    if (!res_b) {
        free_a(res_a);   /* easy to forget */
        return -ENOMEM;
    }

    void *res_c = alloc_c();
    if (!res_c) {
        free_b(res_b);   /* must remember order */
        free_a(res_a);   /* and every prior alloc */
        return -ENOMEM;
    }
    /* … */
}
```

Rust : Drop handles every exit path

```rust
fn my_driver_probe(pdev: &PciDev) -> Result {
    let res_a = ResourceA::alloc()?;
    // If this fails, res_a.drop() is called —
    // even on the ? early return

    let res_b = ResourceB::alloc()?;
    // res_b drops if res_c fails below

    let res_c = ResourceC::alloc()?;

    // All three are freed in reverse order
    // automatically when the function returns
    // — success or failure
    Ok(MyDriver { res_a, res_b, res_c })
}
```

- Cleanup is automatic and deterministic via scope exit, each resource implements `Drop`, ensuring release on all return paths.
- No explicit error-path cleanup logic required.
- Rust turns “developer-managed control flow problem” → “compiler-enforced lifetime and scope rule”
- **Zero `goto` cleanup; chains.** The compiler guarantees cleanup runs on every path.
- Linux has thousands of `goto err_free_XYZ` labels that this eliminates.

4. Borrowing & Lifetimes — Preventing Data Races

Borrowing is Rust's system for temporary access without transferring ownership.
This is implemented via References, which are distinct in Rust compiler view.

Shared (immutable) borrows : `&T`

- Multiple readers can coexist
- None can write while readers exist
- Maps to: read-lock held, RCU read section

```
fn print_all(items: &[DmaEntry]) { /* read-only */ }
let ring = RingBuffer::new(256);
print_all(&ring.entries);  // borrow
print_all(&ring.entries);  // another borrow — fine
// ring is still valid and owned here
```

Data Race = aliasing + mutation + no synchronisation

Exclusive (mutable) borrow : `&mut T`

- **One** writer, **no** concurrent readers
- Maps to: write-lock held, spinlock held

```
fn add_entry(ring: &mut RingBuffer, e: DmaEntry) {
  ring.entries.push(e); // exclusive access
}

let mut ring = RingBuffer::new(256);
add_entry(&mut ring, entry);
// `ring` is fully accessible again after add_entry returns
```

The compiler **proves** that at any point in the program, either **one writer** or **N readers** holds access to any memory location, but never both. This is the data race freedom guarantee.

Lifetimes are the compiler's proof that a reference never outlives the data it points to. Named by the programmer, verified by the borrow checker.

Dangling pointer caught at compile time

```
fn get_ptr() -> &str {           // ← error: missing lifetime
    let local = String::from("DMA buffer");
    &local   // ← would return pointer to stack data
}  // `local` dropped here — pointer would dangle

// Rust requires an explicit lifetime annotation that proves
// the returned reference lives as long as the input:
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
// 'a = "the output lives at least as long as both inputs"
// If this invariant cannot be satisfied, the code does not compile.
```

In kernel drivers: a reference to a `struct device` inside an interrupt handler **must not outlive the device**. Lifetimes encode and verify this invariant statically. No more `dev_hold()` / `dev_put()` mismatches.

The formal argument

A data race requires two conditions simultaneously:

1. **Aliasing** — two pointers to the same memory location
2. **Unsynchronised mutation** — at least one is writing

Ownership rules make both conditions simultaneously impossible in safe code:

- `&mut T` is exclusive ⟶ no aliasing while mutating
- `&T` is immutable ⟶ no mutation while aliased

Therefore: if the program compiles, it has no data races.

This was **formally machine-checked** in Coq by RustBelt (POPL 2018), and extended to C11 relaxed-memory atomics by RustBelt Meets Relaxed Memory (POPL 2020).

Jung et al., POPL 2018; Dang et al., POPL 2020

C

```c
/* Data race — compiles, UB at runtime */
uint64_t shared_counter;

void *thread_a(void *_) {
    shared_counter++;   /* write */
    return NULL;
}
void *thread_b(void *_) {
    shared_counter++;   /* concurrent write */
    return NULL;        /* undefined behaviour */
}
/* gcc: no warning. ThreadSanitizer: catches it
   only if both threads execute concurrently. */
```

Rust

```rust
// Data race — compile error:
let mut counter: u64 = 0;

let t1 = thread::spawn(|| {
    counter += 1; // error[E0373]: closure may
});               // outlive current function,
let t2 = thread::spawn(|| { // but it borrows
    counter += 1; // `counter`, which is owned
});               // by the current function
// Caught BEFORE the binary exists.
// Fix: Arc<AtomicU64> or Arc<Mutex<u64>>
```

5. The Type System as a Security Tool

Two marker traits, zero runtime cost

- `Send`: a type can be safely **moved** to another thread
- `Sync`: a type can be safely **shared** between threads via `&T`
- Both are compile-time properties — no vtable, no runtime check

Automatic, conservative derivation

- `Rc<T>` — **not** `Send` (non-atomic refcount). The compiler refuses to let it cross a thread boundary.
- `Arc<T>` — `Send + Sync` (atomic refcount). Safe to share.
- `Cell<T>` — **not** `Sync` (interior mutability without lock).
- `Mutex<T>` — `Sync` because `lock()` serialises access.

> Any type you write is automatically **not** `Send + Sync` unless all its fields are. This means the **safe default is conservative** — you opt in to thread sharing, you don't opt out of races.

Why this matters for BSP / driver teams

```
// Per-CPU data structure — must not cross CPU boundary
struct PerCpuDmaStats {
    count: u64,
    total_ns: u64,
}
// PerCpuDmaStats contains no Sync interior mutability
// → it is NOT Sync → compiler refuses global shared access

// To share across CPUs: wrap in the correct primitive
use std::sync::Arc;
use kernel::sync::SpinLock;

static SHARED: Arc<SpinLock<PerCpuDmaStats>> = …;

// SpinLock<T> is Sync — its lock() method serialises.
// You cannot access the data without the guard.
// The guard is the only path to the inner T.
```

In C: `/* must hold dma_stats_lock before accessing */` — a comment. In Rust: a compile error if the lock is not held — because the data is not reachable without it.

Null pointer dereferences account for a significant share of kernel crashes. Rust's type system eliminates the possibility by making "might be absent" explicit in the type.

C

```c
/* NULL dereference — common in probe paths */
struct resource *res =
    platform_get_resource(pdev, IORESOURCE_MEM, 0);
/* res might be NULL — easy to forget the check */

void __iomem *base = ioremap(res->start, res->end
                             - res->start + 1);
/* Null dereference if check was skipped → BUG() */
```

Rust

```rust
// Option<T> forces explicit handling of "not found"
let res: Option<Resource> =
    pdev.get_resource(IORESOURCE_MEM, 0);

// Cannot access res.start without handling None first:
let base = match res {
    Some(r) => ioremap(r.start, r.size())?,
    None    => return Err(ErrKind::NoResource),
};
// After the match, `base` is definitely valid.
// There is no raw pointer to dereference unsafely.
```

`Option<&T>` has **identical machine representation** to a nullable pointer — one word, null = `None`, non-null = `Some`. Zero overhead. The safety is entirely in the type, not the runtime. *Rust Reference §3.1 — Niche optimisation*

The C pattern and its failure mode

In C, every `int`-returning function can signal failure via a negative return. The compiler does not warn if the return value is discarded.

```
pci_enable_device(pdev);    /* error silently dropped */
pci_request_regions(pdev, DRV_NAME);   /* same */
request_irq(irq, handler, 0, "drv", dev); /* same */
/* Device may be in undefined state — no indication */
```

Rust's `#[must_use]` + `Result<T, E>`

```
pdev.enable()?;              // ? propagates Err immediately
pdev.request_regions()?;     // compiler proves this only
pdev.request_irq(handler)?;  // runs if enable succeeded
// Each function returns Result<_, Error>.
// Discarding it is a COMPILER WARNING:
// warning[unused_must_use]: unused `Result` that must be used
```

What `?` actually does — no magic

```
// These two are exactly equivalent:
pdev.enable()?;

match pdev.enable() {
    Ok(val)  => val,
    Err(err) => return Err(err.into()),
};
```

- No exceptions — no hidden control flow
- No `longjmp` — no stack unwinding surprise
- The error path is **explicit, local, and grep-able**
- Every early return is a `?` — auditable in code review

> The Linux kernel style guide recommends checking every return value. Rust makes non-checking a **compiler warning by default** — the rule that currently lives in `checkpatch.pl` is instead enforced at compilation.

The C `switch` silent-default problem

```c
enum pcie_speed { GEN1, GEN2, GEN3, GEN4, GEN5 };

uint64_t bandwidth(enum pcie_speed s) {
    switch (s) {
        case GEN1: return 250000000ULL;
        case GEN2: return 500000000ULL;
        case GEN3: return 985000000ULL;
        case GEN4: return 1969000000ULL;
        /* GEN5 added later — falls through to: */
        default:   return 0; /* silent wrong result */
    }
}
/* gcc -Wall: may warn. sparse: may warn.
   Both require the tool to be run AND the warning
   to be treated as an error. Neither is guaranteed. */
```

Rust's exhaustive `match`

```rust
enum PcieSpeed { Gen1, Gen2, Gen3, Gen4, Gen5 }

fn bandwidth(s: PcieSpeed) -> u64 {
    match s {
        PcieSpeed::Gen1 => 250_000_000,
        PcieSpeed::Gen2 => 500_000_000,
        PcieSpeed::Gen3 => 985_000_000,
        PcieSpeed::Gen4 => 1_969_000_000,
        // ↑ forgot Gen5?
        // error[E0004]: non-exhaustive patterns
        //   `PcieSpeed::Gen5` not covered
        //
        // Adding Gen5 to the enum breaks every
        // match that does not handle it — instantly,
        // at every call site, in every crate.
    }
}
```

> There is no `default:` escape hatch by default. When you add a new variant to an enum, the compiler shows you **every** location in the codebase that must be updated.

What unsafe enables

Four capabilities are only available inside `unsafe { }` blocks:

1. Dereference a raw pointer (`*const T`, `*mut T`)
2. Call an `unsafe fn` (including all `extern "C"` FFI)
3. Access or modify a mutable static variable
4. Implement an `unsafe trait` (e.g., `Send`, `Sync` manually)

What unsafe does NOT do

- Does not disable the borrow checker
- Does not disable type checking
- Does not disable lifetime analysis
- Does not disable `#[must_use]`

Only the four capabilities above are relaxed.

The audit argument

```
# Complete audit surface for all driver unsafe code:
grep -rn "unsafe" drivers/my_soc/

# In C: every line is the audit surface.
# There is no equivalent grep.
```

The kernel's three-layer architecture uses this:

```
rust/bindings/     ← all unsafe — auto-generated,
                     reviewed once, never touched
     ↓ wraps
rust/kernel/       ← safe abstractions built on top
     ↓ exposes
drivers/your_ip/   ← pure safe Rust — zero unsafe
                     UAF / race / null proof covers
                     all of your driver code
```

> In C, "how much of this is manually memory-managed?" has no answer. In Rust: **count the unsafe blocks**.

6. Other Key Security-Focused Features

Integer overflow — UB in C, defined in Rust

In C, signed integer overflow is undefined behaviour. The compiler can **legally delete** the overflow check on the assumption it never happens.

```c
// C: signed overflow → UB → compiler may "optimise"
// away the bounds check entirely (documented in GCC docs)
size_t alloc_size = nents * sizeof(struct entry);
if (alloc_size < nents) return -EINVAL; // may be removed!
void *buf = kmalloc(alloc_size, GFP_KERNEL);
```

```rust
// Rust debug build: overflow → controlled panic (BUG())
// Rust release build: wrapping, checked, or saturating
// — explicitly chosen, never undefined:
let alloc_size = nents
    .checked_mul(size_of::<Entry>())
    .ok_or(ErrKind::Overflow)?;
// Returns Err if it overflows — never silent.
```

Panic = explicit abort, not UB

In kernel Rust (`#![no_std]`, custom `#[panic_handler]`), a panic does not unwind — it calls `BUG()` or halts the CPU, exactly like a kernel `BUG_ON()`. There is no hidden exception mechanism.

```rust
#[cfg(not(test))]
#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    // In kernel context: trigger BUG() / halt
    loop {}   // or: call kernel's panic() function
}
```

> Rust panics are **predictable**, **locatable** (they include source location in debug builds), and **never undefined behaviour**. A Rust panic is always intentional; a C signed overflow is always a potential silent time bomb.

C's dangerous default: everything is mutable

```c
void configure_hw(struct hw_config *cfg) {
    /* cfg->base_addr can be modified here
       even if that was not the intent.
       Only `const struct hw_config *` prevents it,
       and const is frequently omitted. */
    cfg->base_addr = 0; // oops — accidental mutation
}
```

Rust's safe default: everything is immutable

```rust
fn configure_hw(cfg: &HwConfig) {
    // cfg.base_addr = 0; // ← error[E0596]:
    //   cannot assign to `cfg.base_addr`,
    //   which is behind a `&` reference
}
// Mutation requires explicit declaration:
fn reconfigure(cfg: &mut HwConfig) {
    cfg.base_addr = NEW_BASE; // only allowed here
}
```

Why this matters for BSP code

- Read-only device tree data, register maps, and firmware blobs are **structurally immutable** — the type prevents accidental writes
- A configuration struct passed to an ISR as `&HwConfig` **cannot** be modified by the ISR — the compiler proves it
- Global immutable data (calibration tables, fuse values) declared as `static` — Rust distinguishes `static T` (immutable, `Sync`-required) from `static mut T` (requires `unsafe` — surfaces immediately in audit)

> **The secure-by-default principle**: mutability must be explicitly requested. The conservative default prevents a class of accidental writes that are otherwise silent in C.

C: intent in comments, verifiable nowhere

```c
/*
 * MUST be called with dma_lock held.
 * dev pointer MUST remain valid for the
 * duration of the DMA operation.
 * Returns negative errno on failure.
 */
int map_dma(struct device *dev,
            struct sg_table *sgt,
            int nents,
            enum dma_data_direction dir);
```

None of these constraints are checked by the compiler. The comment is the only enforcement mechanism.

Rust: intent in the type, checked at every call site

```rust
// "MUST be called with dma_lock held"
// → take a MutexGuard parameter — unobtainable otherwise
fn map_dma<'lock>(
    _guard:  &'lock MutexGuard<DmaState>, // proof of lock
    dev:     &'lock Device,               // must outlive guard
    sgt:     &mut SgTable,
    dir:     DmaDirection,                // typed enum, not int
) -> Result<SgMapping<'lock>, DmaError>  // typed error
// The comment is now the type signature.
// Violations are compile errors, not runtime bugs.
```

> Every invariant encoded in a Rust type signature is **checked at every call site in every crate that uses it** — including crates that were written years later by engineers who never read the comment.

7. Compiler Checks : Beyond Memory safety:

Memory safety (previous sections)

- No use-after-free
- No dangling references
- No data races
- No null dereferences (`Option<T>` instead of `NULL`)
- No uninitialized reads (all fields must be initialised)

Type system

- Integer overflow in debug builds ⟶ panic (not UB)
- Exhaustive `match` — all enum variants handled

```
match direction {
  DmaDir::ToDevice   => { … },
  DmaDir::FromDevice => { … },
  // error if DmaDir::Bidirectional not handled
}
```

Error handling

- `Result<T, E>` — ignoring an error is a **compiler warning**

```
#[must_use = "this Result must be handled"]
fn map_dma(...) -> Result<SgTable, DmaError> { … }

map_dma(dev, sg, nents); // warning: unused Result
// The kernel C equivalent: silently dropping -ENOMEM
```

- `Option<T>` — no null, no null-deref

Unsafe quarantine

- `unsafe { }` blocks are **explicitly marked**
- `grep -r "unsafe"` gives the **complete** audit surface
- Safe code cannot call `unsafe` functions accidentally

> In C, all of the above require separate tools: ASAN, UBSAN, sparse, Coccinelle, clang-tidy, GCC sanitizers — none of them are mandatory, and none are exhaustive.

8. The Broader Rust Ecosystem

Unified toolchain — no separate install decisions

Tool	Role (and C equivalent)
`cargo`	build, test, bench, publish — replaces Make + CMake + pkg-config
`rustfmt`	auto-formatter — replaces `clang-format` debates
`clippy`	semantic linter — replaces Coverity / sparse for Rust code
`rustdoc`	doc generation from `///` comments — replaces Doxygen
`cargo-generate`	kickstart a new project using git repo as template
`cargo-expand`	Rust code: after all macros have been expanded.
`rust-analyzer`	LSP: autocomplete, inline errors, go-to-definition — IDE-native

The ecosystem signal: Rust is reshaping every layer

- **These slides** — written in **Typst**, which is itself written in Rust
- **ripgrep** (`rg`) — faster `grep`, written in Rust *github.com/BurntSushi/ripgrep*
- **fd** — faster `find`
- **Ruff** — Python linter, 10–100× faster than pylint *astral.sh/ruff*
- **uv** — Python package manager, replaces pip
- **swc** — JS/TS compiler, 70× faster than Babel
- **Cloudflare Pingora** — HTTP proxy serving 1 trillion requests/day *Cloudflare Blog 2022*
- **AWS Firecracker** — microVM hypervisor powering Lambda + Fargate *NSDI 2020*
- **Android 16** — kernel 6.12, `ashmem` allocator in Rust — production on millions of devices

> The meta-point for this audience: **the tool generating these slides is written in Rust**. The language has escaped "systems niche" and is reshaping the entire software toolchain stack.

Additional topics that are for those who want to go down the rabbit hole:

- **Traits**: Rust's explicit code interfaces, defining shared behavior across different data types.
- **Generics**: Compile-time templates that allow you to write algorithms that work with multiple data types without code duplication, evaluated entirely at build time.
- **Trait-bounds**: Compile-time constraints on generics, allowing you to tell the compiler: 'This generic function only works on types that implement a specific hardware interface or trait.'
- **Smart pointers**: Custom data structures that act like pointers but wrap raw memory with automatic lifecycle tracking, like managing a reference-counted memory region.
- **iterators**: Highly optimized, composable pointer-traversal abstractions that let you loop through arrays or ring buffers safely without raw index pointer arithmetic.
- **macros**: Code-generation tools that compile down at build time, allowing you to write highly expressive code without incurring any runtime or performance overhead.
- **attributes**: Declarative metadata attached to your code, used for things like conditional compilation for different processor targets or forcing explicit struct packing layout alignment.

```
# 1. Install rustup (manages Rust toolchain versions)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"

# 2. Verify installation
rustc --version   # rustc 1.XX.0 (...)
cargo --version   # cargo 1.XX.0 (...)

# 3. Install useful components
rustup component add rustfmt clippy rust-analyzer

# 4. First project
cargo new hello-driver
cd hello-driver
cargo build
cargo run
cargo clippy      # linter
cargo fmt         # formatter
cargo test        # unit tests

# 5. Recommended learning path for C/kernel developers
# The Rust Programming Language (free): https://doc.rust-lang.org/book/
# Rustlings (interactive exercises): https://rustlings.cool
# Rust by Example: https://doc.rust-lang.org/rust-by-example/
# Comprehensive Rust (Google, Android focus): https://google.github.io/comprehensive-rust/
```

8.4 Key references

Formal verification

- Jung, R., Jourdan, J-H., Krebbers, R., Dreyer, D. **"RustBelt: Securing the Foundations of the Rust Programming Language."** POPL 2018. https://plv.mpi-sws.org/rustbelt/popl18/
- Dang, H-H., Jourdan, J-H., Kaiser, J-O., Dreyer, D. **"RustBelt Meets Relaxed Memory."** POPL 2020.

Industry data

- Microsoft MSRC. "A Proactive Approach to More Secure Code." Gavin Thomas, 2019.
- Google Project Zero. "Memory Safety Issues in Chrome." 2020.
- Android Security Blog. "Memory Safety in Android." 2021. https://security.googleblog.com
- CISA. "The Case for Memory Safe Roadmaps." 2023. https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps

Language specification and learning

- The Rust Reference. https://doc.rust-lang.org/reference/
- The Rust Programming Language (Klabnik & Nichols). https://doc.rust-lang.org/book/
- Comprehensive Rust (Google). https://google.github.io/comprehensive-rust/

Performance

- Paczos, K. "The Speed of Rust vs C." https://kornel.ski/rust-c-speed/ 2023.
- Compiler Explorer. https://godbolt.org

Production deployments

- Cloudflare. "How We Built Pingora." 2022. https://blog.cloudflare.com/how-we-built-pingora
- Agache et al. "Firecracker." NSDI 2020.